# Project 6 — Relationship Manager Copilot for Wealth Management (Custom Python · LangGraph)

## 1. Project Overview

### Goal:

Build a multi-tool agentic assistant for wealth-management relationship managers (RMs) using:

- OpenAI LLMs (GPT-4o / GPT-4.1) for reasoning and synthesis
- OpenAI Embeddings for policy/research retrieval
- ChromaDB / FAISS for product, policy, and research knowledge
- LangGraph for multi-step, multi-tool agent orchestration

The system must help RMs prepare for client interactions by:

- Summarizing a client's portfolio and risk profile
- Retrieving relevant market/research and product information (RAG)
- Checking suitability/compliance constraints before any recommendation
- Producing a grounded, compliant talking-points brief with citations
- Escalating anything requiring licensed advice or human sign-off

## 2. High-Level Architecture

### Components:

- Knowledge / Vector Store — product guide, suitability policy, research notes, compliance rules
- Tools — portfolio lookup (DB/API), market-data tool, suitability checker, RAG retriever
- LangGraph Agent — plan → gather (portfolio + research) → check suitability → synthesize → review
- Structured Output + Guardrails — Pydantic brief, compliance gate, citations
- Frontend / API — Streamlit or FastAPI RM dashboard (optional)

## 3. Data Model Design (Pydantic + Vector Store)

### Pydantic: ClientBrief

| Field | Type | Description |
|---|---|---|
| **client_id** | str | Client identifier |
| **risk_profile** | enum | Conservative / Balanced / Growth / Aggressive |
| **portfolio_summary** | str | Holdings + allocation summary |
| **recommendations** | list[Reco] | idea, rationale, suitability, citations |

| | | |
|---|---|---|
| **compliance_status** | enum | Cleared / Needs Review / Blocked |
| **talking_points** | list[str] | RM-ready discussion points |

### Vector store metadata

- doc_id, type (product/policy/research), date, source, sensitivity

## 4. Ingestion Pipeline (LangChain)

### Step 1 — Load Documents

Load product guides, suitability/compliance policy, and research notes (PDF/DOCX/TXT). A minimum of 10 source documents across types must be ingested.

### Step 2 — Preprocessing & Chunking

- Structure-aware chunking; attach type/date/sensitivity metadata

### Step 3 — Embedding & Indexing

- Embed and store in ChromaDB/FAISS with metadata filters for entitlement and freshness

## 5. Agentic Pipeline (LangGraph)

### Graph Logic

1. plan: decompose the RM request (e.g., 'prep for client X review')
2. gather_portfolio: tool call to fetch holdings + risk profile
3. gather_research: hybrid RAG over product/research knowledge
4. check_suitability: validate ideas against suitability & compliance policy
5. synthesize: produce a grounded ClientBrief with citations
6. review_gate: if licensed-advice or blocked ⟶ escalate to a human; else return brief

### Note: Use ReAct-style tool calling and bounded loops (max steps).

## 6. Structured Output Using Pydantic

The agent must return a validated ClientBrief. Each recommendation must carry a rationale, a suitability assessment, and citations to the supporting product/policy/research chunk.

## 7. Safety & Guardrails

- Grounding + citations for every recommendation; no ungrounded advice
- Compliance/suitability gate before any recommendation is surfaced
- Never give personalized investment advice that requires a license — escalate
- Entitlement filtering on knowledge (sensitivity) and client data
- Temperature control, bounded agent steps, full trace/audit logging

- Clear disclaimers: decision support for RMs, not automated advice

## 8. End-to-End Agent (LangGraph)

Assemble the LangGraph StateGraph with shared state (client context, retrieved evidence, draft brief), conditional edges for the compliance gate, a human-in-the-loop interrupt, and observability hooks (LangSmith) for tracing each tool call and retrieval.

## 9. Deliverables (Capstone Requirements)

### Part A — Engineering

- Knowledge ingestion + hybrid RAG retriever with metadata filters
- Portfolio-lookup, market-data, and suitability-checker tools
- LangGraph agent: plan → gather → check → synthesize → review
- Pydantic ClientBrief with citations
- Compliance gate + human-in-the-loop escalation
- Guardrails + LangSmith tracing

### Part B — Analytics

- Evaluate retrieval relevance and citation coverage on a golden set
- Measure suitability-gate precision (blocks unsuitable ideas)
- Faithfulness/groundedness of recommendations (LLM-as-judge / RAGAS)
- Compare single-shot vs multi-hop retrieval on complex requests

### Part C — Final UI

- Streamlit RM dashboard: enter client → generate grounded brief
- or a Notebook playground with interactive client selection

## 10. Final Submission Requirements

Participants must submit a presentation containing all results, visuals, and relevant screenshots. Additionally, they must provide a working demo of the code that supports all the following queries/scenarios:

- 'Prepare talking points for client C-204's quarterly review' → grounded ClientBrief
- 'Is fund XYZ suitable for a conservative client?' → suitability check + citation
- 'Summarize the portfolio risk for client C-204' → portfolio summary
- Recommendation requiring licensed advice → escalation, not auto-advice
- Restricted research for an unentitled RM → filtered out (access control)

The submission must include: source code repository, requirements.txt/environment file, the evaluation notebook with metrics, sample input data, and a live walk-through